



第四章 判断与断言(上海-悠悠)

第四章 判断与断言(上海-悠悠)

4.1 page页面 expect 断言的几种方式

- 4.1.1 页面断言
- 4.1.2 断言url
- 4.1.3 使用示例

4.2 expect 常用的断言方法

- 4.2.1 expect 使用
- 4.2.2 断言可见与不可见
- 4.2.3 断言是否被选中

4.3 如何判断元素是否存在

- 4.3.1 locator 定位元素
- 4.3.2 query_selector 定位

4.4 判断页面元素状态checkbox和radio

- 4.4.1 常用的元素判断方法
- 4.4.2 locator 定位后判断元素
- 4.4.3 page对象调用的判断方法

4.8 如何对比 2 张图片相似度

- 4.8.1 Ctrl+c/v 借代码
- 4.8.2 cv2.resize() 使用
- 4.8.3 使用示例

- [网易云视频课程](#)

4.1 page页面 expect 断言的几种方式

当打开一个页面的时候，需要断言是否是期望的页面

PageAssertions类提供断言方法，可用于在测试中对页面状态进行断言。

4.1.1 页面断言

主要有四个断言方法

- to_have_title

- not_to_have_title
- to_have_url
- not_to_have_url

to_have_title() 确保页面具有给定的标题。

```
import re
from playwright.sync_api import expect
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

# ...
expect(page).to_have_title(re.compile(r".*checkout"))
```

参数

- title_or_reg_exp 预期的标题或正则表达式。
- timeout （可选） 超时时间， 默认5000ms

not_to_have_title 与expect(page).to_have_title()相反。

```
expect(page).not_to_have_title(title_or_reg_exp)
expect(page).not_to_have_title(title_or_reg_exp, **kwargs)
```

4.1.2 断言url

to_have_url 确保页面导航到给定的 URL。

```
import re
from playwright.sync_api import expect

# ...
expect(page).to_have_url(re.compile(".*checkout"))
```

not_to_have_url 与expect(page).to_have_url()相反。

```
expect(page).not_to_have_url(url_or_reg_exp)
expect(page).not_to_have_url(url_or_reg_exp, **kwargs)
```

4.1.3 使用示例

传统断言方式，先获取到值，再判断相等/不相等/包含 ...

这种获取值，再去判断的方式有缺陷：

获取的是当时页面上的状态，页面上状态可能会发生改变，比如点一个按钮，状态由 "未完成" 变成 "已完成"，可能会有延迟情况。

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False)
    context = browser.new_context()
    page = context.new_page()
    page.goto("https://www.baidu.com")

    title = page.title()
    print(title)
    assert title == "百度一下，你就知道"
    print(page.url)
    assert page.url == "https://www.baidu.com/"
```

playwright 推荐的断言方式，使用expect

```
from playwright.sync_api import sync_playwright, expect

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False)
    context = browser.new_context()
    page = context.new_page()
    page.goto("https://www.baidu.com")

    title = page.title()
    print(title)
    print(page.url)
    expect(page).to_have_title("百度一下，你就知道")
    expect(page).to_have_url("https://www.baidu.com/")
```

expect 断言 超时时间默认是5秒，在5秒内判断达不到预期结果，就报超时异常。

4.2 expect 常用的断言方法

playwright 提供了一个 expect方法 用于断言

4.2.1 expect 使用

断言	描述
<code>expect(locator).to_be_checked()</code>	Checkbox is checked
<code>expect(locator).to_be_disabled()</code>	Element is disabled
<code>expect(locator).to_be_editable()</code>	Element is enabled
<code>expect(locator).to_be_empty()</code>	Container is empty
<code>expect(locator).to_be_enabled()</code>	Element is enabled
<code>expect(locator).to_be_focused()</code>	Element is focused
<code>expect(locator).to_be_hidden()</code>	Element is not visible
<code>expect(locator).to_be_visible()</code>	Element is visible
<code>expect(locator).to_contain_text()</code>	Element contains text
<code>expect(locator).to_have_attribute()</code>	Element has a DOM attribute
<code>expect(locator).to_have_class()</code>	Element has a class property
<code>expect(locator).to_have_count()</code>	List has exact number of children
<code>expect(locator).to_have_css()</code>	Element has CSS property
<code>expect(locator).to_have_id()</code>	Element has an ID
<code>expect(locator).to_have_js_property()</code>	Element has a JavaScript property
<code>expect(locator).to_have_text()</code>	Element matches text
<code>expect(locator).to_have_value()</code>	Input has a value
<code>expect(locator).to_have_values()</code>	Select has options selected

断言	描述
<code>expect(page).to_have_title()</code>	Page has a title
<code>expect(page).to_have_url()</code>	Page has a URL
<code>expect(api_response).to_be_ok()</code>	Response has an OK status

4.2.2 断言可见与不可见

`to_be_visible()`使用示例

```
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

from playwright.sync_api import expect

locator = page.locator('.my-element')
expect(locator).to_be_visible()
```

4.2.3 断言是否被选中

`to_be_checked()`使用示例

```
from playwright.sync_api import expect

locator = page.get_by_label("Subscribe to newsletter")
expect(locator).to_be_checked()
```

4.3 如何判断元素是否存在

playwright 如何判断某个元素是否存在？

4.3.1 locator 定位元素

使用 `locator` 定位元素，不管元素存不存在，都会返回一个 `locator` 对象，可以用到 `count()` 方法统一元素的个数，如果元素个数是 0，那么元素就不存在

```

"""
判断元素存在
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/
"""

from playwright.sync_api import sync_playwright

with sync_playwright() as pw:
    browser = pw.chromium.launch()
    page = browser.new_page()

    page.goto("https://www.baidu.com/")

    # 元素存在
    loc1 = page.locator("id=kw")
    print(loc1)
    print(loc1.count())

    # 元素不存在
    loc2 = page.locator('id=yoyo')
    print(loc2)
    print(loc2.count())

```

运行结果

```

<Locator frame=<Frame name= url='https://www.baidu.com/'> selector='id=kw'>
1
<Locator frame=<Frame name= url='https://www.baidu.com/'> selector='id=yoyo'>
0

```

locator 是定位当前页面上的元素，不会自动等待，如果用click等方法结合使用，会自动去等待元素处于可点击状态。

4.3.2 query_selector 定位

ElementHandle 表示页内 DOM 元素。ElementHandles 可以使用page.query_selector()方法创建。

Locator和ElementHandle之间的区别在于后者指向特定元素，而 Locator 捕获如何检索该元素的逻辑。

元素存在返回元素句柄，元素不存在返回None

```
# 元素存在
loc1 = page.query_selector('#kw')
print(loc1) # JSHandle@node

# 元素不存在
loc2 = page.query_selector('#yoyo')
print(loc2) # None
```

也可以用query_selector_all 复数定位方式返回一个list

```
# 元素存在
loc1 = page.query_selector_all('#kw')
print(loc1) # [<JSHandle preview=JSHandle@node>]

# 元素不存在
loc2 = page.query_selector_all('#yoyo')
print(loc2) # []
```

对于用户来说，元素存不存在其实不重要，用户只关注元素的状态是可见还是不可见

4.4 判断页面元素状态checkbox和radio

在操作元素之前，可以先判断元素的状态。判断元素操作状态也可以用于断言。

4.4.1 常用的元素判断方法

page对象调用的判断方法, 传一个selector 定位参数

- page.is_checked(selector: str) # checkbox or radio 是否选中
- page.is_disabled(selector: str) # 元素是否可以点击或编辑
- page.is_editable(selector: str) # 元素是否可以编辑
- page.is_enabled(selector: str) # 是否可以操作
- page.is_hidden(selector: str) # 是否隐藏
- page.is_visible(selector: str) # 是否可见

locator 对象调用的判断方法

- locator.is_checked()
- locator.is_disabled()

- locator.is_editable()
- locator.is_enabled()
- locator.is_hidden()
- locator.is_visible()

元素句柄 的判断方法

- element_handle.is_checked()
- element_handle.is_disabled()
- element_handle.is_editable()
- element_handle.is_enabled()
- element_handle.is_hidden()
- element_handle.is_visible()

元素句柄（element_handle）是通过page.query_selector()方法调用返回的ElementHandle，这种一般不常用。

关于元素句柄和locator 定位的区别这篇有介绍<https://www.cnblogs.com/yoyoketang/p/17190635.html>

4.4.2 locator 定位后判断元素

locator 对象调用的判断方法

- locator.is_checked()
- locator.is_disabled()
- locator.is_editable()
- locator.is_enabled()
- locator.is_hidden()
- locator.is_visible()

is_checked() 用于判断checkbox or radio 的状态是否被选中


```

<div>
  <label>性别:
    <input type="radio" name="sex" id="man" checked>男
    <input type="radio" name="sex" id="woman">女
    <input type="radio" name="sex" id="no" disabled>人妖
  </label>
</div>
<div>
  <label>标签:
    <input type="checkbox" id="a1"> 旅游
    <input type="checkbox" id="a2">看书
    <input type="checkbox" id="a3" checked >学习
    <input type="checkbox" id="a4" checked disabled>学python
  </label>
</div>

```

性别: ☒男 ☐女 ☐人妖
 标签: ☐ 旅游 ☐看书 ☒学习 ☒学python

代码示例

```

print(page.locator('#man').is_checked()) # checked
print(page.locator('#man').is_enabled())
print(page.locator('#no').is_checked())
print(page.locator('#no').is_enabled()) # disabled

```

返回结果

```

True
True
False
False

```

4.4.3 page对象调用的判断方法

page对象调用的判断方法, 传一个selector 定位参数

- page.is_checked(selector: str) # checkbox or radio 是否选中
- page.is_disabled(selector: str) # 元素是否可以点击或编辑
- page.is_editable(selector: str) # 元素是否可以编辑
- page.is_enabled(selector: str) # 是否可以操作

- `page.is_hidden(selector: str)` # 是否隐藏
- `page.is_visible(selector: str)` # 是否可见

使用示例

```
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/
print(page.is_checked('#a3'))
print(page.is_enabled('#a3'))
print(page.is_checked('#a4'))
print(page.is_enabled('#a4'))
```

运行结果

```
True
True
True
False
```

总的来说有2种方式判断元素 `page.is_xx()` 和 `locator.is_xxx()`

4.8 如何对比 2 张图片相似度

在做 web 自动化的时候，有些场景需要去判断页面上的图片与预期的图片是否一样，或者判断图片有没正确的加载出来，需用到图片对比。

如果你之前接触过airtest，那么你应该知道它是专业搞图片对比的，所以我们应该去那借点代码过来！

环境准备：

```
pip install opencv-python
pip install numpy
```

4.8.1 Ctrl+c/v 借代码

找到 `Lib\site-packages\airtest\aircv` 目录下的 `cal_confidence.py` 文件，就是我们要借的代码了

<< 本地磁盘 (E:) > python36 > Lib > site-packages > airtest > aircv 搜索"aircv"				
	名称	修改日期	类型	大小
Personal	_pycache_	2021/6/3 22:57	文件夹	
	init.py	2019/4/22 19:39	Python File	1 KB
	aircv.py	2021/4/6 15:14	Python File	5 KB
	cal_confidence.py	2021/4/6 15:14	Python File	2 KB
Is	error.py	2019/12/26 16:06	Python File	2 KB
	keypoint_base.py	2021/4/28 17:34	Python File	17 KB
	keypoint_matching.py	2019/4/22 19:39	Python File	2 KB
	keypoint_matching_contrib.py	2021/4/30 17:56	Python File	6 KB
	sift.py	2018/8/28 11:38	Python File	16 KB
	template.py	2019/4/22 19:39	Python File	5 KB
	template_matching.py	2019/4/22 19:39	Python File	6 KB
C:)	utils.py	2021/4/6 15:14	Python File	3 KB

上海-悠悠

```

#!/usr/bin/env python
# -*- coding: utf-8 -*-

"""These functions calculate the similarity of two images of the same size."""

import cv2
from .utils import img_mat_rgb_2_gray

def cal_ccoeff_confidence(im_source, im_search):
    """求取两张图片的可信度，使用TM_CCOEFF_NORMED方法."""
    # 扩展置信度计算区域
    im_search = cv2.copyMakeBorder(im_search, 10,10,10,10,cv2.BORDER_REPLICATE)

    im_source, im_search = img_mat_rgb_2_gray(im_source), img_mat_rgb_2_gray(im_search)
    res = cv2.matchTemplate(im_source, im_search, cv2.TM_CCOEFF_NORMED)
    min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(res)

    return max_val

def cal_rgb_confidence(img_src_rgb, img_sch_rgb):
    """同大小彩图计算相似度."""
    # 扩展置信度计算区域
    img_sch_rgb = cv2.copyMakeBorder(img_sch_rgb, 10,10,10,10,cv2.BORDER_REPLICATE)
    # 转HSV强化颜色的影响
    img_src_rgb = cv2.cvtColor(img_src_rgb, cv2.COLOR_BGR2HSV)
    img_sch_rgb = cv2.cvtColor(img_sch_rgb, cv2.COLOR_BGR2HSV)
    src_bgr, sch_bgr = cv2.split(img_src_rgb), cv2.split(img_sch_rgb)

    # 计算BGR三通道的confidence，存入bgr_confidence:
    bgr_confidence = [0, 0, 0]
    for i in range(3):
        res_temp = cv2.matchTemplate(src_bgr[i], sch_bgr[i], cv2.TM_CCOEFF_NORMED)
        min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(res_temp)
        bgr_confidence[i] = max_val

    return min(bgr_confidence)

```

img_mat_rgb_2_gray 对应代码

```
def img_mat_rgb_2_gray(img_mat):  
    """  
    Turn img_mat into gray_scale, so that template match can figure the img data.    "print(type(i  
    return cv2.cvtColor(img_mat, cv2.COLOR_BGR2GRAY)
```

主要用到2个方法

- cal_ccoeff_confidence 灰度对比，先把2张图片弄成黑白色，再去对比
- cal_rgb_confidence 彩色对比，这个对彩色的颜色要求更精准。

正常情况下，如果是对比相似度，用cal_ccoeff_confidence 就可以了。

如果你要判断2个图片完全一样，那就用cal_rgb_confidence 彩色对比。

4.8.2 cv2.resize() 使用

使用语法：cv2.resize(src, dsize, dst=None, fx=None, fy=None, interpolation=None)

- src 源图像
- dsize 输出图像的大小
- fx width方向的缩放比例
- fy height方向的缩放比例
- interpolation 这个是指定插值的方式

因为对比2张图片先要把图片大小设置为一致，使用cv2.resize()方法处理原始的图片，用于对比。

4.8.3 使用示例

准备3张图片用于测试



```
if __name__ == '__main__':  
    # 对比2个图片  
    img1 = cv2.resize(cv2.imread('images/liu1.png'), (100, 100))  
    img2 = cv2.resize(cv2.imread('images/liu2.png'), (100, 100))  
    res1 = cal_ccoeff_confidence(img1, img2)  
    print(res1) # 0.32  
    img3 = cv2.resize(cv2.imread('images/liu3.png'), (100, 100))  
    res2 = cal_ccoeff_confidence(img2, img3)  
    print(res2) # 0.75
```

网易云视频课程

网易云视频完整课程地址<https://study.163.com/course/courseMain.htm?courseId=1213382811&share=2&shareId=480000002230338>

